Sobel Filter on FPGA

Authors:
WhisperingRock

Professor:

Class:
Quarter:
September 8, 2026

Contents

Chapter 1

Theory of the Sobel Operator

1.1 Grayscale Operator

This implementation requires that the RBG image (TODO img type) be flattened into a single intensity value (a grayscale value). This can be done with a few methods.

- Average :

$$p = \frac{R + G + B}{3}$$

- ITU-R B.709 Luminance

$$p = 0.2126R + 0.7152G + 0.0722B$$

I opt'd to use Luminance for its accepted standard.

1.2 Sobel Operator

Uses two 3x3 kernels which are convolved with the original (grayscale) image to calculate approximations of the derivatives. Each kernel, one for horizontal change and one for vertical change, performs a low pass Gaussian filtering [1 2 1] as well a discrete differentiation [+1 0 -1] on each pixel of the image.

$$\mathbf{G_x} = \begin{bmatrix} +1 & 0 & -1 \\ +2 & 0 & -2 \\ +1 & 0 & -1 \end{bmatrix} = \begin{bmatrix} 1 \\ 2 \\ 1 \end{bmatrix} \begin{bmatrix} +1 & 0 & -1 \end{bmatrix}$$

$$\mathbf{G_y} = \begin{bmatrix} +1 & +2 & +1 \\ 0 & 0 & 0 \\ -1 & -2 & -1 \end{bmatrix} = \begin{bmatrix} +1 \\ 0 \\ -1 \end{bmatrix} \begin{bmatrix} 1 & 2 & 1 \end{bmatrix}$$

Consider the resulting components :

- Magnitude :

$$\mathbf{G} = \sqrt{\mathbf{G_x}^2 + \mathbf{G_y}^2}$$

- Direction of gradient :

$$\mathbf{\Theta} = \arctan\left(\frac{\mathbf{G_y}}{\mathbf{G_x}}\right)$$

The results of the operator produce a cheap edge detector that performs some smoothing while retaining a DC gain of 1. For the scope of this project, I'm only using the magnitude.

Chapter 2

System Architecture

Chapter 3

FPGA IP

3.1 Line Buffer

The LineBuffer is designed to . . .

- incrementally collect individual input pixel values into the internal storage, using *DIN_ AVAIL* and *DIN*, on each clock cycle. *DIN_ AVAIL* enables writes while also moving the write pointer to the next position.
- always output a 3 pack of neighbor pixels to *DOUT*, only updating and shifting on a clock'd *DOUT_ AVAIL*.
- naturally rollover internal readPtr and writePtr to 512 count (pixels) or reset to zero using *RST*. One might consider the buffer circular using the rollover.

Notes:

- Need at least three writes at or ahead of readPtr before using output data to avoid reading junk.

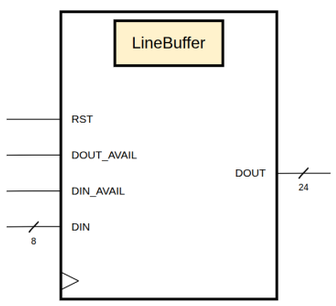


Figure 3.1: LineBuffer Block

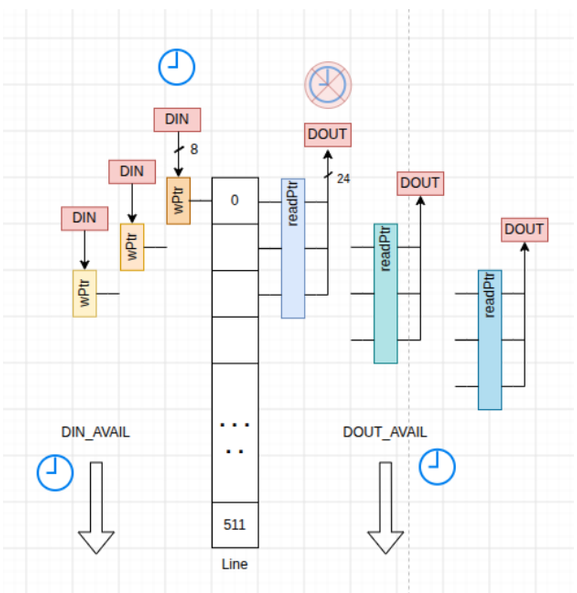


Figure 3.2: LineBuffer Visualized

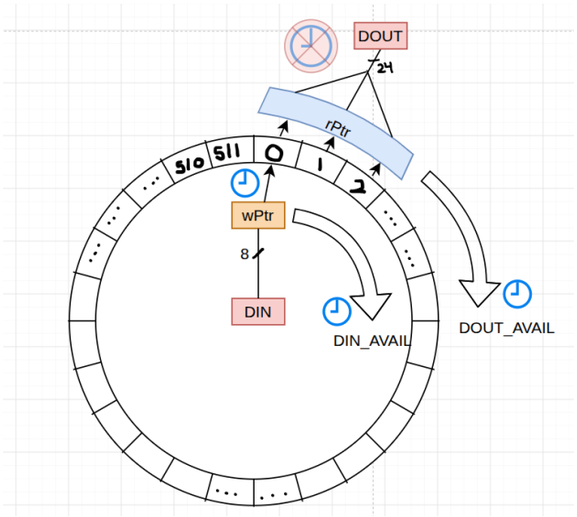


Figure 3.3: LineBuffer Visualized as Circular

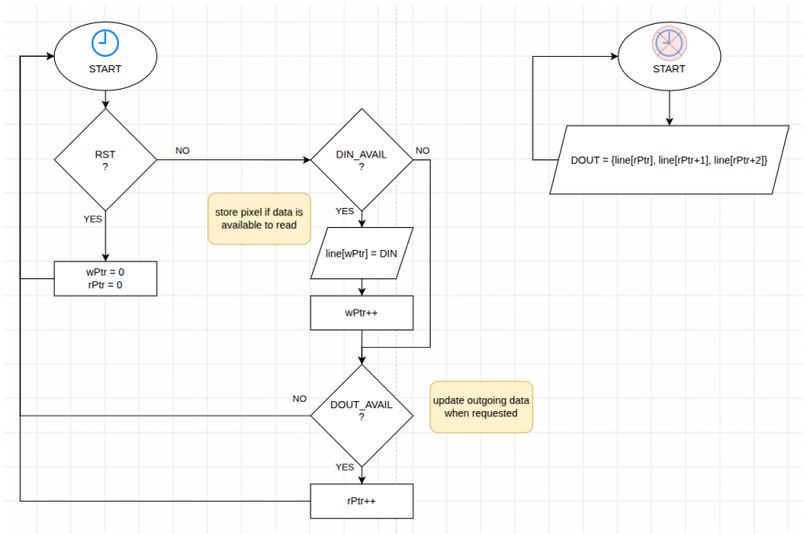


Figure 3.4: LineBuffer Flowchart

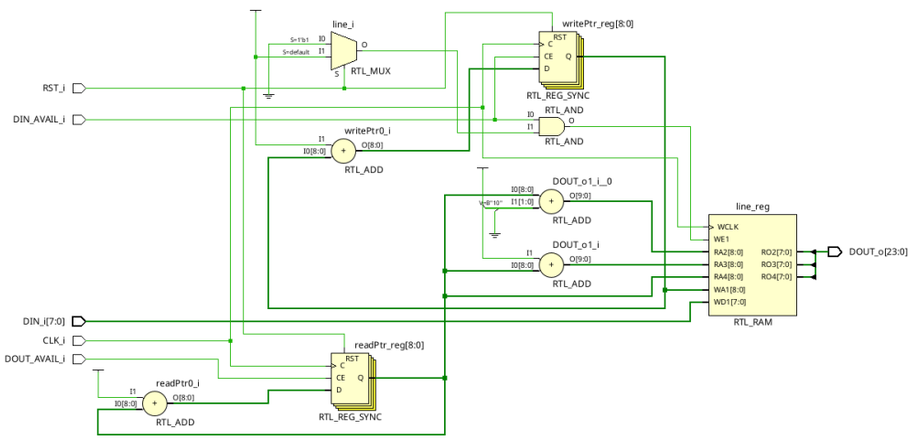


Figure 3.5: LineBuffer RTL

Code :

- `LineBuffer.sv` 7.1
- `LineBuffer_tb.sv` 7.2

3.2 Convolution

The Convolution module is designed to . . .

- Perform a single matrix multiplication as part of convolving the image with the kernel filter.

The kernels used were :

Unity Gain Box Blurr

$$k = \frac{1}{9}\begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$$

Sobel X-axis

$$k = \begin{bmatrix} 1 & 0 & -1 \\ 2 & 0 & -2 \\ 1 & 0 & -1 \end{bmatrix}$$

Sobel Y-axis

$$k = \begin{bmatrix} 1 & 2 & 1 \\ 0 & 0 & 0 \\ -1 & -2 & -1 \end{bmatrix}$$

3.2.1 Design

Always:

$$sum = \sum_{i=0}^{8} m[i] * k[i]$$

CLK:

$$\texttt{Result} = \begin{cases} \texttt{Result} & \text{, if } \overline{\texttt{MATRIX\_VALID}} \\ \begin{cases} 0 & \text{, if } sum < 0 \\ 255 & \text{, if } sum > 255 \\ \frac{sum}{div} & \text{, else} \end{cases} & \text{, if } \texttt{MATRIX\_VALID} \end{cases}$$

$$\texttt{RESULT\_VALID} = \texttt{MATRIX\_VALID}$$

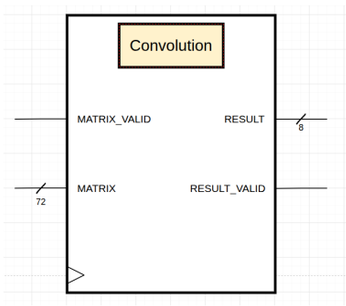


Figure 3.6: Convolution Block

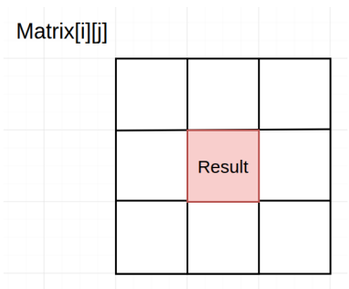


Figure 3.7: Convolution Visualized

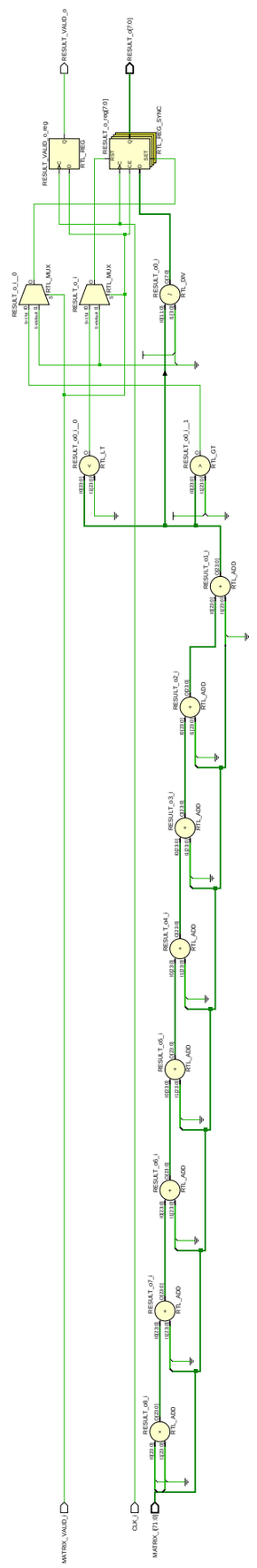


Figure 3.8: Convolution RTL

Code :

- `Convolution.sv` 7.3
- `Convolution_tb.sv` 7.4

3.3 Image Control Unit

The IMU is designed to :

- Orchestrate which LineBuffer is to receive incoming pixel data through the use of counters and indexers `pixelCounter` and `currWrLineBuff`. `pixelCounter` is the current pixel for the row and `currWrLineBuff` indicates which LineBuffer is recieving the incoming data.
- Orchestrate which LineBuffers are contributing to the exported 3x3 matrix window through the use of counters and indexers `windowCounter` and `currRdLineBuff`. `windowCounter` is the window's left-most pixel in reference to a row of pixels. Said another way, incrementing this variable slides the 3x3 window right by one pixel and resets after reaching 511 pixels. `currRdLineBuff` controls which combination of LineBuffer outputs are used to create the window s.t. incrementing this variable shifts the 3x3 window down by one pixel.
- Ensure there are at least three rows of pixel data in the buffer before sliding the output window. This is tracked using `currBufferedPixels` and the FSM logic.

NOTE : I really need to use UVM encapsulation/sanitization for testing this module. The read and write pointers do not line up and there is no way to erase the memory contents of the line buffers with reset. That would probably take a considerable effort to do such a thing...

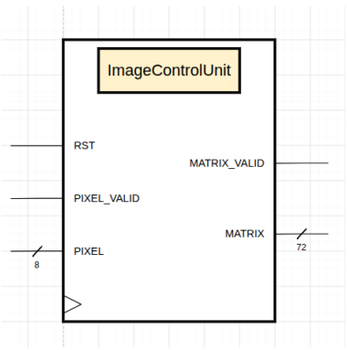


Figure 3.9: IMU Block

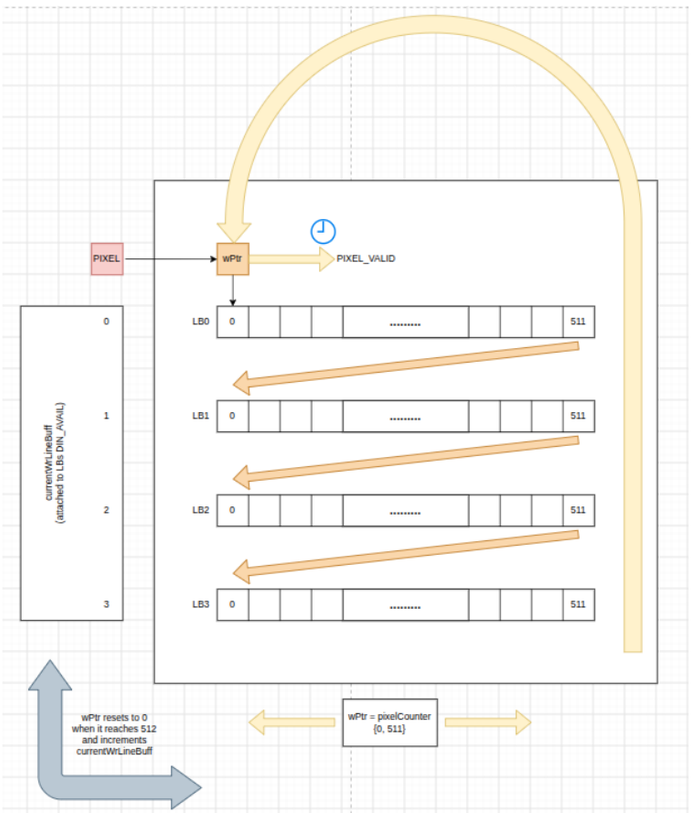


Figure 3.10: IMU Data Importing Visualized

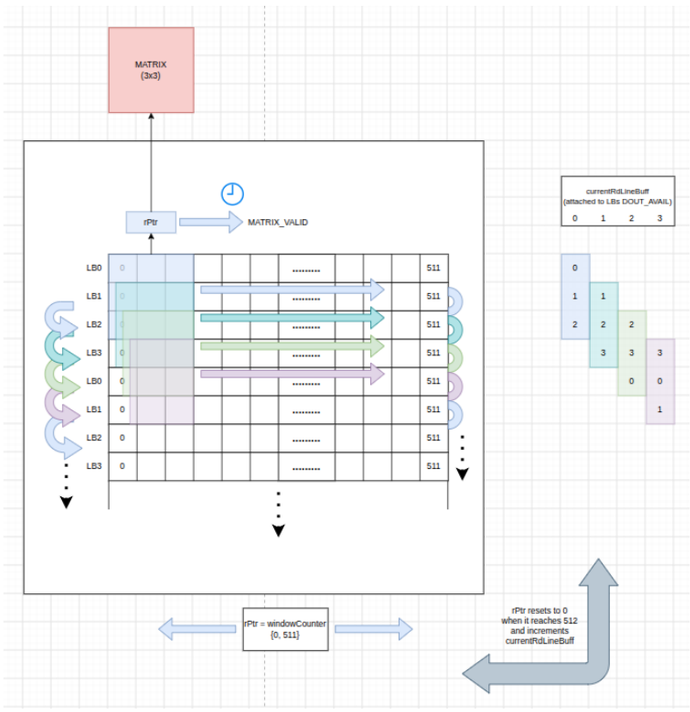


Figure 3.11: IMU Data Exporting Visualized

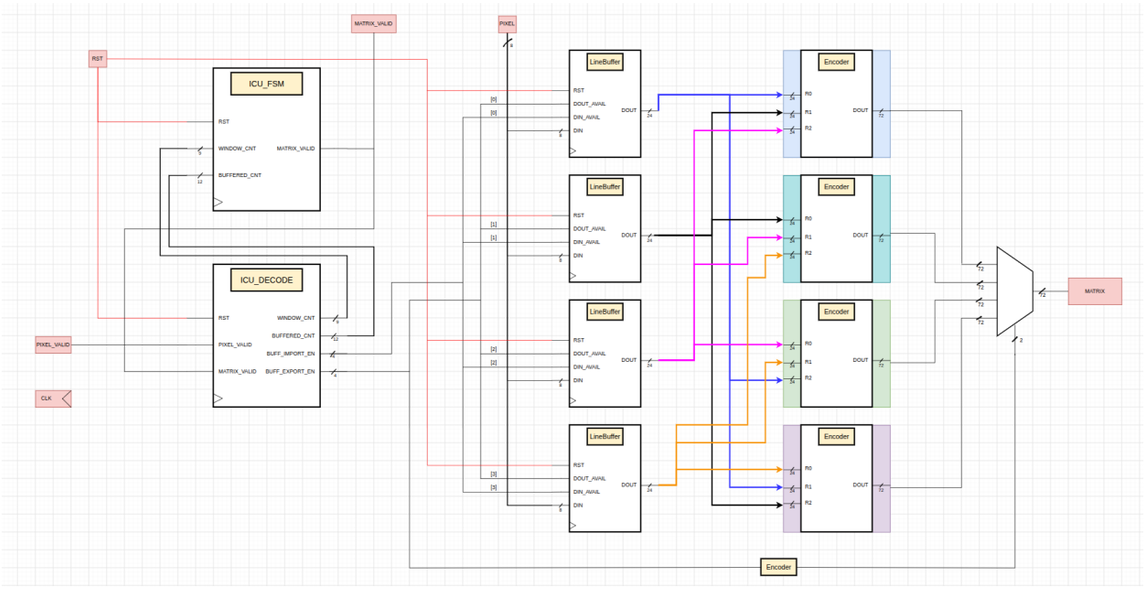


Figure 3.12: IMU Visualized in another way

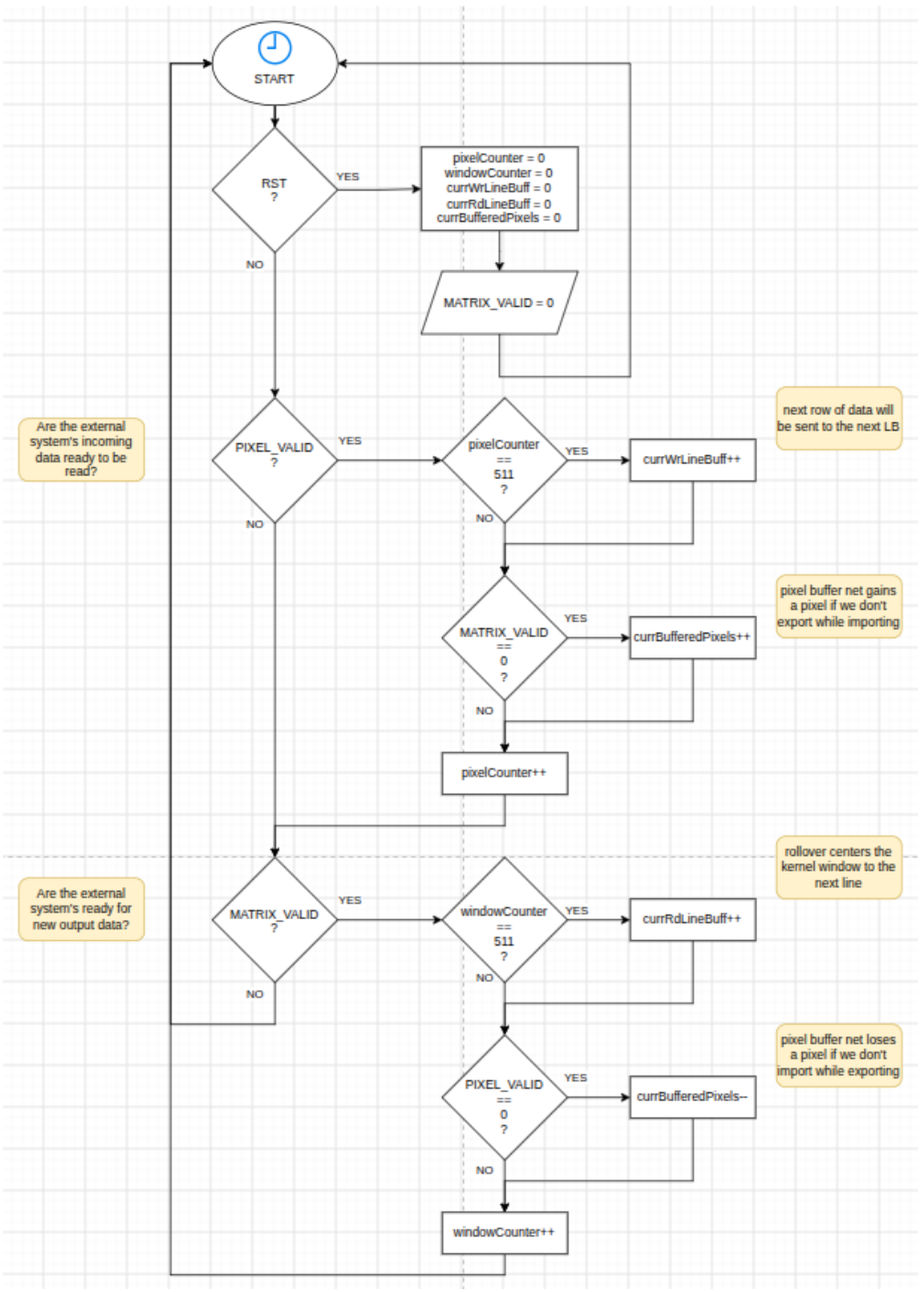


Figure 3.13: IMU CLK Sync'd Flowchart

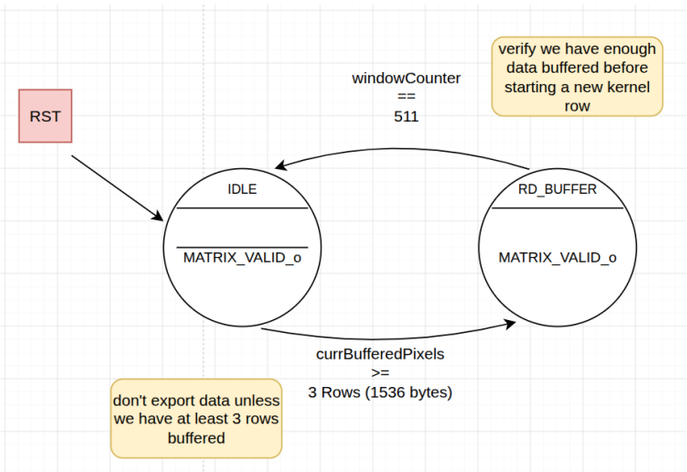


Figure 3.14: IMU FSM

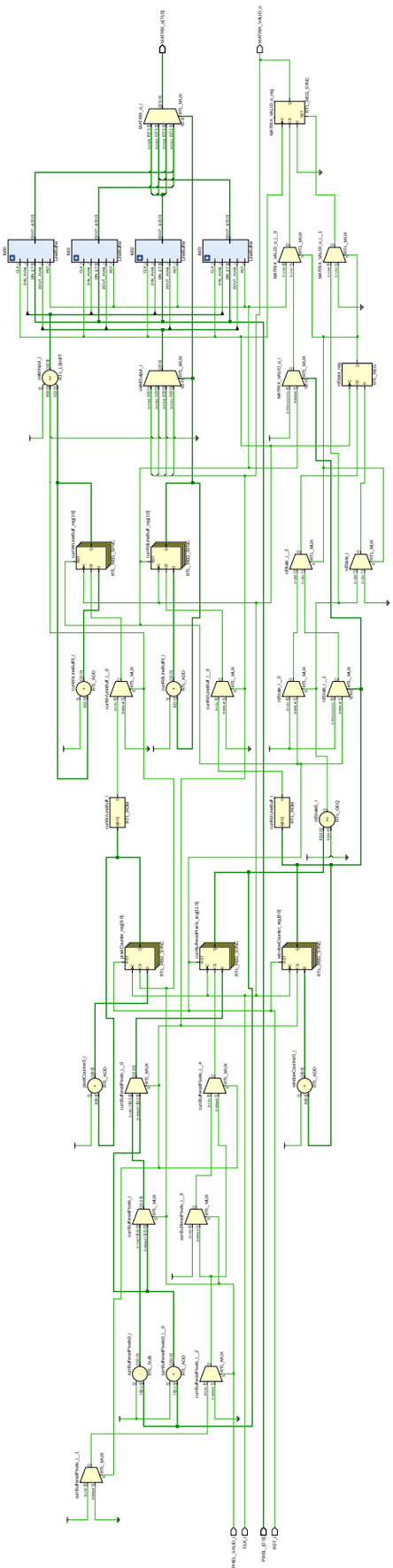


Figure 3.15: IMU RTL

Code :

- `ImageControlUnit.sv` 7.5
- `ImageControlUnit_tb.sv` 7.6

3.4 Image Control Top

The Image Control Top module is designed to :

- ...

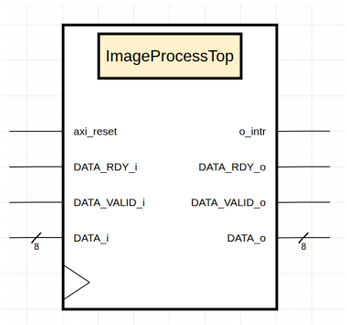


Figure 3.16: IM Top Block

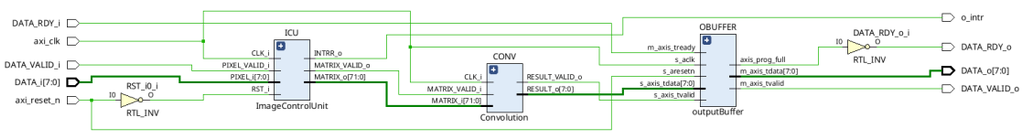


Figure 3.17: IM Top Hardware

3.4.1 Output Buffer

The module now has a 2-depth FIFO output buffer, supplied by Vivado IP, but why? Well, the FIFO is useful because the filter pipeline could produce result pixels at a different rate than the downstream could accept them. Without the buffer, the convolution block could overwrite its previous result before it is read. Fortunately for us, this scenario is more likely than the entire pipeline stalling out but that could still occur.

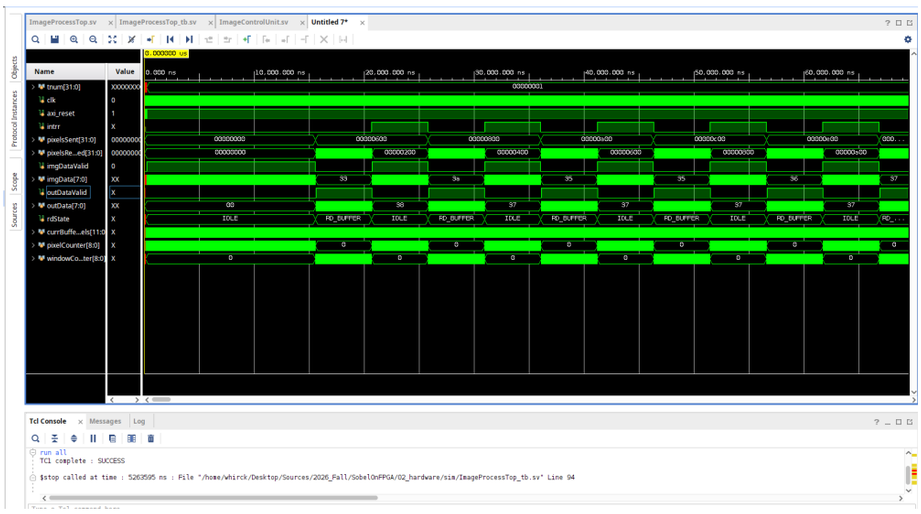
While the current simulation is not connecting the full indicator, this will be useful for regulating the pipeline back pressure.

3.4.2 AXI Bus

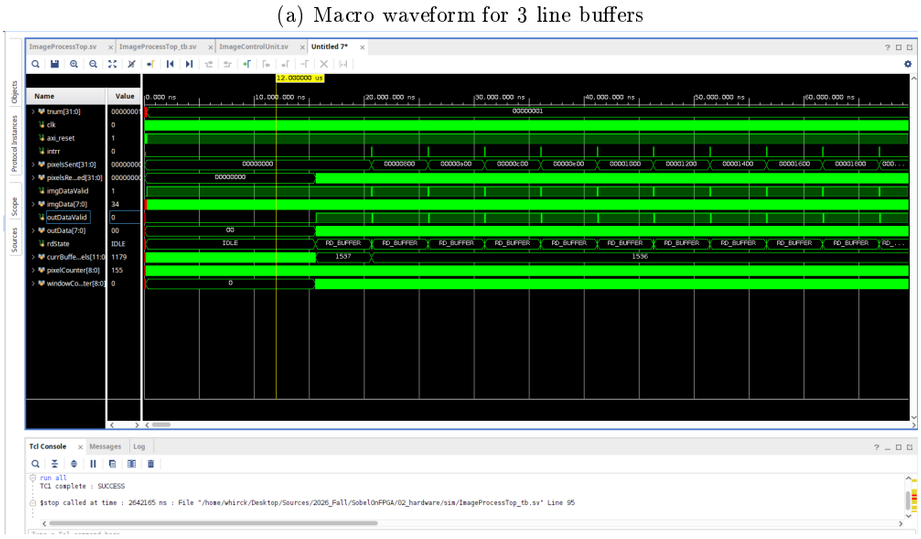
3.4.3 Design Questions

Q: How many buffers should we initially fill?

The model needs at least three buffers filled to start the convolution and we only have four line buffers to fill so some potential options are pre-filling three or all four line buffers. Lets compare the sim results side by side :



(a) Macro waveform for 3 line buffers



(b) Macro waveform for 4 line buffers

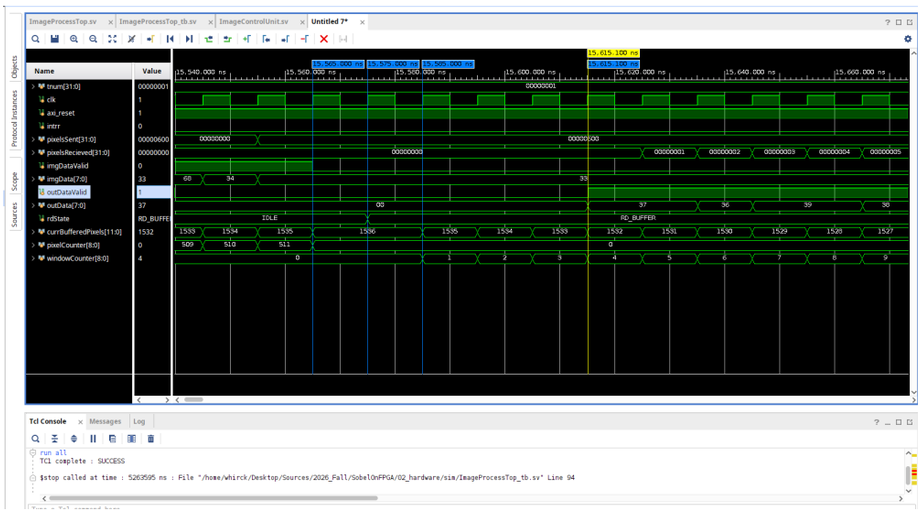
Figure 3.18: Simulation between 3 and 4 prefilled LineBuffers

Observations between waveforms :

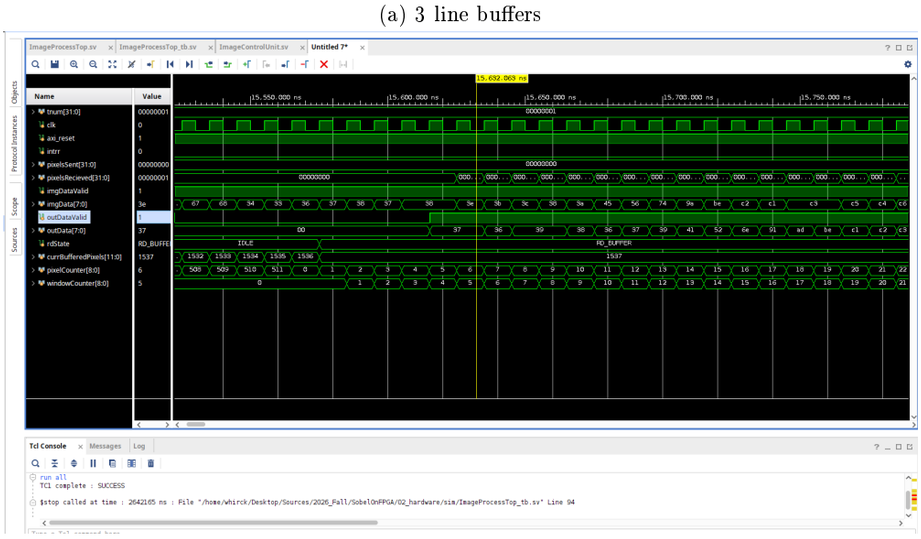
- **Timing** : It took the 3LBs 5.26 `ms` and 2.64 `ms` for 4LB to complete the block blurr algorithm. It almost took twice as long to do the entire process based off of a single prefill. Thats big.
- **Read/Write** : For 3LB, we are not importing in new values while writing nor are we exporting data while importing. This is also reflected in the states where we reside in one logic state for 512 reads or writes but not both. For 3LB, it seems we are always waiting to read or write but never both.

  For 4LB, it appears we briefly go into the `IDLE` state but largely remain in the `RD_BUFFER` state. This somehow allows us to import AND export data at the same time, likely cutting down our compute time in the process.

Given such large gap in performance between the two options, lets take a closer look at what is going on here.



(a) 3 line buffers



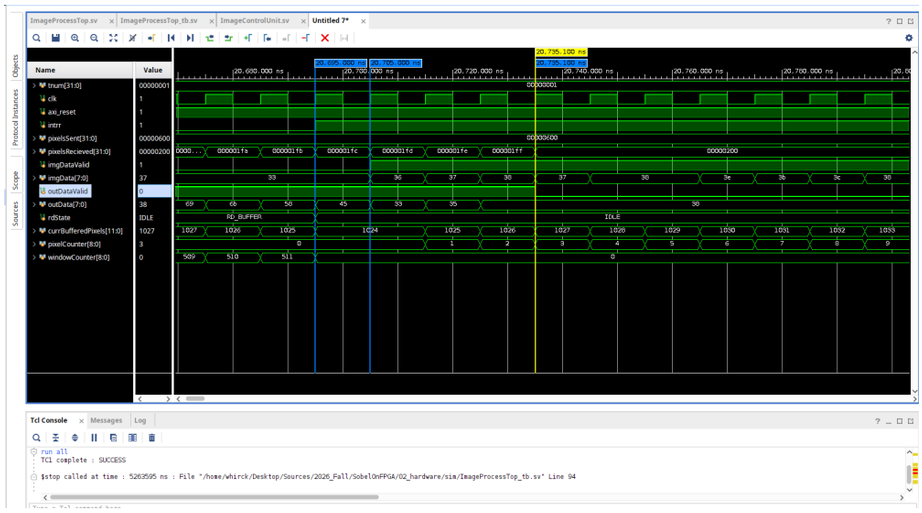
(b) 4 line buffers

Figure 3.19: Simulation start between 3 and 4 prefilled LineBuffers

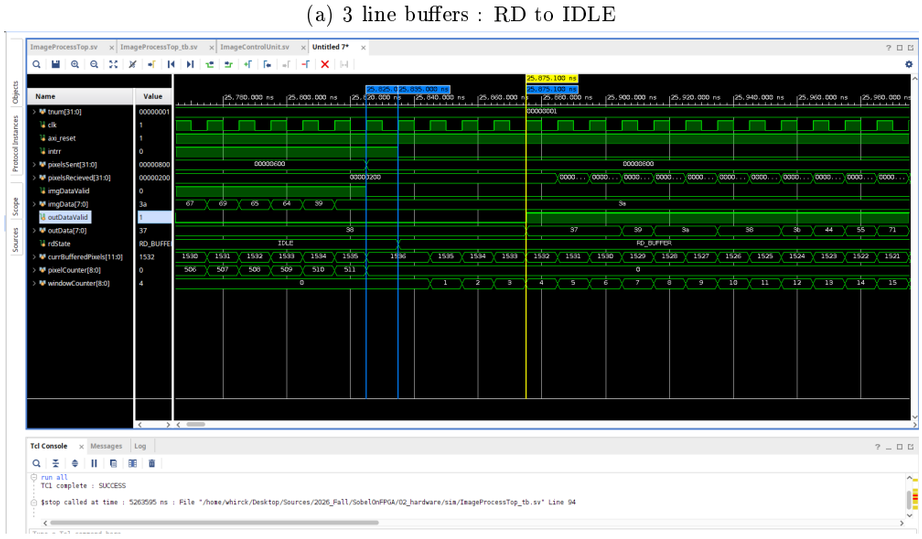
Observations between these starting waveforms:

- Both begin outputting result pixel data and reach the `RD_BUFFER` state.
- Only 4LB accepts `imgData` by keeping `imgDataValid` high but 3LB lowers this signal. This is our 4th row being written to the buffer.
- Both waveforms reach a net buffer count of three rows (1536) in `currBufferedPixels` but the 3LB doesn't retain this number. 3LB decreases in net pixel count because it is no longer importing data while exporting, something of a triangle wave of net bufferd pixels.
- No interrupt signals are present for both signals during this startup.

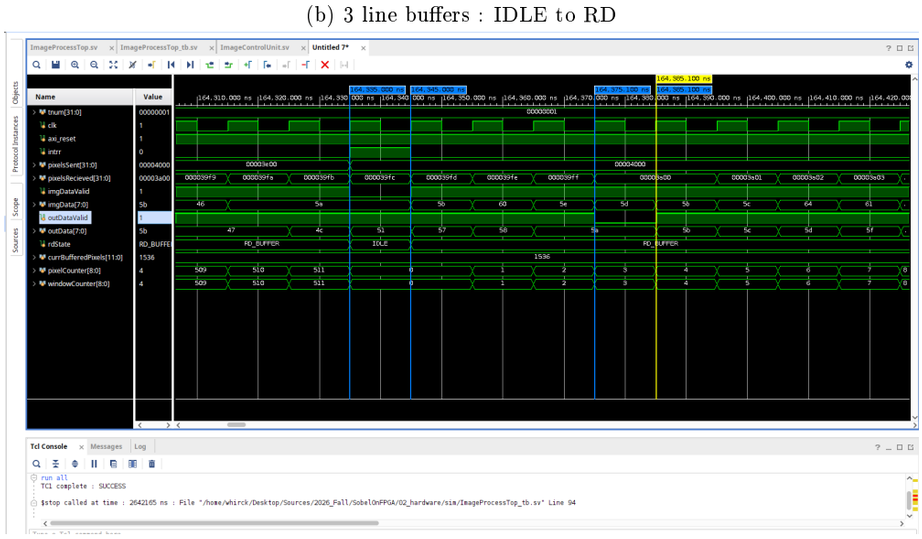
Lastly, lets consider the waveforms at the crossover between FSM logic.



(a) 3 line buffers : RD to IDLE



(b) 3 line buffers : IDLE to RD



(c) 4 line buffers

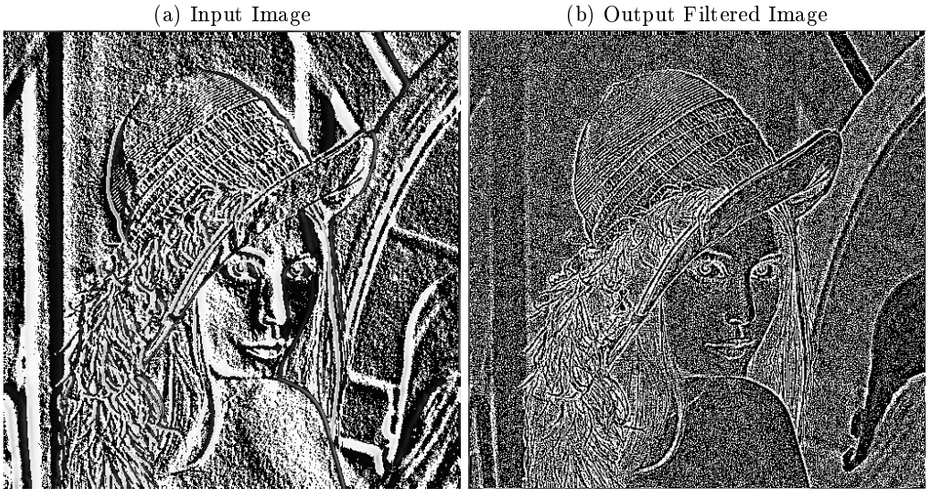
Figure 3.20: Simulation start between 3 and 4 prefilled LineBuffers

3.4.4 Testing

The kernel used was the box blur which averages all neighboring pixels with a gain of 1.

$$K = \frac{1}{9}\begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$$

(a) Input Image (b) Output Filtered Image



(c) Difference Image (shift unfixed) (d) Difference Image (shift fixed)

Figure 3.21: I/O of image

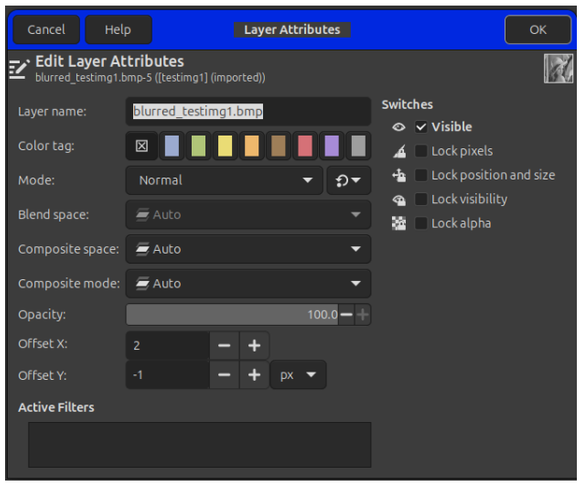


Figure 3.22: Offset on Output Image

Observations

- Output image (b) is blurred
- Output image (b) has a black pixel boarder on the top and right sides of the picture.
- Difference image (d) gives a much better depiction of the high-frequency details that were removed than image (c). The pixels are also uniform throughout the image as well, which is not the case in (c).

3.4.5 Result Questions

Q:Why was the entire output image shifted?

Chapter 4

Results

Chapter 5

Analysis

Chapter 6

Conclusion

Chapter 7

Appendix

7.1 SystemVerilog

7.1.1 Line Buffer

Listing 7.1: LineBuffer.sv

```
`timescale 1ns / 1ps
//////////////////////////////////////////////////////////////////////////////////
// Company:
// Engineer:
//
// Create Date: 08/31/2026 09:16:29 AM
// Design Name:
// Module Name: LineBuffer
// Project Name:
// Target Devices:
// Tool Versions:
// Description:     Gathers a single row (512) of pixel data, one pixel at a time,
//                  and outputs a single row (3) for a 3x3 kernel. The input_avail
//                  signal indicates the next pixel is ready to be read and the
//                  output_avail indicates the downstream module is ready for a
//                  kernel window slide.
//
//                  How to use :
//                      - Fill the row largely unrestricted but prevent rollover until ready for new row
//                      - Only slide kernel row when sobel operation has finished that that position
//                      - reset can also be used rather than rollover to start a new row/window
//
// Dependencies:
//
// Revision:
// Revision 0.01 - File Created
// Additional Comments:
//
//////////////////////////////////////////////////////////////////////////////////


module LineBuffer
#(parameter int unsigned PWIDTH     = 512,
            int unsigned PHEIGHT    = 512,
            int unsigned WPTR_BIT   = 9,    // 2^9 = 512
            int unsigned RPTR_BIT   = 9
)
(
    input logic             CLK_i,
    input logic             RST_i,
    input logic     [7:0]   DIN_i,
    input logic             DIN_AVAIL_i,    // input data ready to be read/sent to this mod
    input logic             DOUT_AVAIL_i,   // output data ready to be read/recieved from this mod
    output logic    [23:0]  DOUT_o          // packing 3 byte/pixels into a single word. Not using packed
          array to reduce dimension
);

    // ~~~~ local params/alloc ~~~~
    logic [7:0]             line [0:PWIDTH-1];          // line contains 512 (bytes) pixels
    logic [WPTR_BIT - 1:0]  writePtr;                   // 9 bits covers 512 pixel width (2^9)
    logic [RPTR_BIT - 1:0]  readPtr;


    // ~~~~ sync logic ~~~~
    always_ff @(posedge CLK_i)
    begin

        // ~~ priority 0 : reset r/w ptrs to zero place ~~
        if(RST_i) begin
            writePtr    <= 0;
            readPtr     <= 0;
        end

        // ~~  priority 1 : read/write ~~
        else begin

            // ~ Store pixel i in row/line when available ~
            if(DIN_AVAIL_i == 1'b1) begin
                line[writePtr]  <= DIN_i;
                writePtr        <= writePtr + 1'b1;
            end

            // ~ Slide the kernel window (this row in particular) right 1 when prompted ~
            if(DOUT_AVAIL_i == 1'b1) begin
                readPtr         <= readPtr + 1'b1;
```

```
            end
        end

    end

    // ~~~~ comb logic ~~~~
    assign DOUT_o = {line[readPtr], line[readPtr + 2'b01], line[readPtr + 2'b10]};  // pack 3 pixels for kernel row

endmodule
```

Listing 7.2: LineBuffer_tb.sv

```
`timescale 1ns / 1ps
//////////////////////////////////////////////////////////////////////////////////
// Company:
// Engineer:
//
// Create Date: 08/31/2026 10:33:22 AM
// Design Name:
// Module Name: LineBuffer_tb
// Project Name:
// Target Devices:
// Tool Versions:
// Description:
//
// Dependencies:
//
// Revision:
// Revision 0.01 - File Created
// Additional Comments:
//
//////////////////////////////////////////////////////////////////////////////////


module LineBuffer_tb();

    // ~~~~ imports ~~~~
    import tb_utils_pkg::*;

    // ~~~~ local param/allocs ~~~~
    // ~~ ports ~~
    logic                   clk;
    logic                   reset;
    logic   [7:0]           din;
    logic                   din_avail;
    logic                   dout_avail;
    logic   [23:0]          dout;

    // ~~ consts ~~
    localparam int unsigned PIXELWIDTH  = 8;    // smaller for ease of testing
    localparam int unsigned PIXELHEIGHT = 8;
    localparam int unsigned WPTR_BIT    = $clog2(PIXELWIDTH);
    localparam int unsigned RPTR_BIT    = $clog2(PIXELWIDTH);

    // ~~ testing ~~
    testcase                    tc;
    logic   [31:0]              tnum;
    bit     [PIXELWIDTH-1:0][7:0]   arr;


    // ~~~~ module instances ~~~~
    LineBuffer
    #(
        .PWIDTH(PIXELWIDTH),
        .PHEIGHT(PIXELHEIGHT),
        .WPTR_BIT(WPTR_BIT),
        .RPTR_BIT(RPTR_BIT)
    )
    UUT
    (
        .CLK_i(clk),                // 1'b
        .RST_i(reset),              // 1'b
        .DIN_i(din),                // 8'b
        .DIN_AVAIL_i(din_avail),    // 1'b
        .DOUT_AVAIL_i(dout_avail),  // 1'b
        .DOUT_o(dout)               // 24'b
    );

    // ~~~~ heartbeat (10ns) ~~~~
    always begin
        #5;
        clk <= !clk;
    end

    // ~~~~ testing ~~~~
    initial begin

        // ~~ class instances ~~
        tc              = new();

        // ~~ defaults ~~
        clk             = 1'b0;
        reset           = 1'b1;
        din             = 8'h00;
        din_avail       = 1'b0;
        dout_avail      = 1'b0;
        #100;
        reset           = 1'b0;
        #5;


        // ~~ TC1 ~~
        tc.new_test("Filling Line");
        tnum            = tc.get_testnum();

            arr         = 64'h08_07_06_05_04_03_02_01;
```

```
95              din_avail   = 1'b1;
96
97              tc.print_subtest("2/3 of writes");
98              din         = arr[0];
99              #5;
100             assert(dout === {arr[0], 16'hXX_XX})            else tc.err("DOUT error");
101             #5;
102
103             din         = arr[1];
104             #5;
105             assert(dout === {arr[0], arr[1], 8'hXX})        else tc.err("DOUT error");
106             #5;
107
108             tc.print_subtest("Disable write");
109             din_avail   = 1'b0;
110             din         = arr[2];
111             #5;
112             assert(dout === {arr[0], arr[1], 8'hXX})        else tc.err("DOUT error");
113             #5;
114
115             tc.print_subtest("Re-enable write for 3/3 writes");
116             din_avail   = 1'b1;
117             #5;
118             assert(dout === {arr[0], arr[1], arr[2]})       else tc.err("DOUT error");
119             #5;
120
121             tc.print_subtest("Full buffer prevents new data");
122             din         = arr[3];
123             #5;
124             assert(dout === {arr[0], arr[1], arr[2]})       else tc.err("DOUT error");
125             #5;
126         tc.test_done();
127
128
129         // ~~ TC2 ~~
130         tc.new_test("Reset ptrs");
131         tnum            = tc.get_testnum();
132             reset           = 1'b1;
133             din_avail       = 1'b0;
134             dout_avail      = 1'b0;
135             #10;
136
137             // overwrite first 3 spots with zeros
138             reset           = 1'b0;
139             din_avail       = 1'b1;
140             din             = 8'h00;
141             #30;
142             assert(dout === {24'h00_00_00})                 else tc.err("DOUT error");
143         tc.test_done();
144
145         // ~~ TC3 ~~
146         tc.new_test("Fill to top, then empty");
147         tnum            = tc.get_testnum();
148             // place writePtr back to zero
149             reset           = 1'b1;
150             #10;
151             reset           = 1'b0;
152             din_avail       = 1'b1;
153             dout_avail      = 1'b0;
154
155             // fill
156             din         = arr[0];
157             #10;
158             din         = arr[1];
159             #10;
160             din         = arr[2];
161             #10;
162
163             // fill
164             for(int i = 3; i < PIXELWIDTH; i++) begin
165                 $display("|-%d", i);
166                 din         = arr[i];
167                 #5;
168                 assert(dout === {arr[0], arr[1], arr[2]})   else tc.err("DOUT static error");
169                 #5;
170             end
171
172             // empty
173             din_avail       = 1'b0;
174             dout_avail      = 1'b1;
175             for(int i = 3; i < PIXELWIDTH; i++) begin
176                 $display("|-%d", i);
177                 #5;
178                 assert(dout === {arr[i-2], arr[i-1], arr[i]})   else tc.err("DOUT emptying error");
179                 #5;
180             end
181
182             #5;
183             assert(dout === {arr[PIXELWIDTH-2], arr[PIXELWIDTH-1], arr[0]})     else tc.err("DOUT 2/3
                    full error");
184             #10;
185             assert(dout === {arr[PIXELWIDTH-1], arr[0], arr[1]})                else tc.err("DOUT 1/3
                    full error");
186             #10;
187             assert(dout === {arr[0], arr[1], arr[2]})                           else tc.err("DOUT empty
                    error");
188             #5;
189
190         tc.test_done();
191
192
193         // ~~ TC4 ~~
194         tc.new_test("R/W synch'd rollover");
195         tnum            = tc.get_testnum();
196
197             reset           = 1'b1;
198             #10;
199             reset           = 1'b0;
```

```
            din_avail       = 1'b1;
            dout_avail      = 1'b0;

            for(bit[7:0] i = 0; i < PIXELWIDTH; i = i + 1'b1) begin
                din = i;
                #10;
            end

            din = 8'hFF; // should be placed into the first idx if rollover is correct
            #5;
            assert(dout === {8'hFF, 8'h01, 8'h02})          else tc.err("DOUT error");
            #5;



        tc.test_done();



    end


endmodule
```

7.1.2 Convolution

Listing 7.3: Convolution.sv

```
`timescale 1ns / 1ps
//////////////////////////////////////////////////////////////////////////////////
// Company:
// Engineer:
//
// Create Date: 08/31/2026 07:34:11 PM
// Design Name:
// Module Name: Convolution
// Project Name:
// Target Devices:
// Tool Versions:
// Description:
//
// Dependencies:
//
// Revision:
// Revision 0.01 - File Created
// Additional Comments:
//                      Input Mat LSB is element zero!!!!
//////////////////////////////////////////////////////////////////////////////////


module Convolution
#(
    parameter logic         FILTER = 0
)
(
    input logic                         CLK_i,
    input logic     [71:0]              MATRIX_i,               // 3x3 = 9 pixels, 8 bits each
    input logic                         MATRIX_VALID_i,         // input ready for import
    output logic    [7:0]               RESULT_o,
    output logic                        RESULT_VALID_o          // result ready for export
);

    // ~~~~ local param/alloc/const ~~~~

    localparam signed   [8:0]       pixelmin = 0;
    localparam signed   [8:0]       pixelmax = 255;

    // box blurr
    logic signed        [23:0]      sum_comb;
    logic signed        [23:0]      temp_result;
    logic               [15:0]      div;
    assign                          div = (FILTER == 0) ? 16'd9 : 16'd1;

    localparam logic signed [7:0]   K_BOX [0:8] =
    '{
        8'd1, 8'd1, 8'd1,
        8'd1, 8'd1, 8'd1,
        8'd1, 8'd1, 8'd1
    };

    // sobel
    logic signed        [23:0]      sum_x, sum_y;
    logic signed        [23:0]      tmp_x, tmp_y;
    localparam signed   [23:0]      sobel_thresh = 4000;
    localparam logic signed [7:0]   K_SOBEL_X [0:8] =
    '{
        1, 0, -1,
        2, 0, -2,
        1, 0, -1
    };
    localparam logic signed [7:0]   K_SOBEL_Y [0:8] =
    '{
         1,  2,  1,
         0,  0,  0,
        -1, -2, -1
    };



    // ~~~~ async logic ~~~~
    // Note : will perform only once til new data flows in
    always_comb begin

        // ~~ sanitize between kernel operations ~~
        sum_comb    = '0;
        sum_x       = '0;
        sum_y       = '0;
```

```
80
81          // ~~ box blurr filter ~~
82          if(FILTER == 0) begin
83              // ~~ compute kernel sum per CLK ~~
84              for(int i = 0; i < 9; i++) begin
85                  sum_comb += K_BOX[i] * $signed({1'b0, MATRIX_i[i*8 +: 8]});
86              end
87          end
88
89          // ~~ sobel filter ~~
90          else begin
91              // ~~ compute kernel sum per CLK ~~
92              for(int i = 0; i < 9; i++) begin
93                  sum_x += K_SOBEL_X[i] * $signed({1'b0, MATRIX_i[i*8 +: 8]});
94                  sum_y += K_SOBEL_Y[i] * $signed({1'b0, MATRIX_i[i*8 +: 8]});
95              end
96          end
97
98
99      end
100
101
102     // ~~~~ sync logic ~~~~
103     always_ff @(posedge CLK_i) begin
104
105         RESULT_VALID_o <= MATRIX_VALID_i;                                   // result likely ready if
                input is
106
107         if(MATRIX_VALID_i) begin
108
109             // box filter
110             if(FILTER == 0) begin
111                 temp_result <= sum_comb / div;
112
113                 // pixel clipping
114                 if(temp_result < pixelmin)          begin   RESULT_o <= 8'd0;           end // floor clip
115                 else if(temp_result > pixelmax)     begin   RESULT_o <= pixelmax;       end // or ceil
                        clip
116                 else                                begin   RESULT_o <= temp_result;    end // or apply
                        gain factor
117             end
118
119             // sobel filter
120             else begin
121
122                 tmp_x <= $signed(sum_x) * $signed(sum_x); // Gx ^ 2
123                 tmp_y <= $signed(sum_y) * $signed(sum_y); // Gy ^ 2
124                 temp_result <= $signed(tmp_x) + $signed(tmp_y); // Gx^2 + Gy^2
125
126                 // Avoid square root and use threshold instead
127                 if(temp_result < sobel_thresh)  begin   RESULT_o <= pixelmin;   end
128                 else                            begin   RESULT_o <= pixelmax;   end
129
130             end
131         end
132     end
133
134
135 endmodule
```

Listing 7.4: Convolution_tb.sv

```
1  `timescale 1ns / 1ps
2  //////////////////////////////////////////////////////////////////////////////////
3  // Company:
4  // Engineer:
5  //
6  // Create Date: 09/01/2026 08:47:08 AM
7  // Design Name:
8  // Module Name: Convolution_tb
9  // Project Name:
10 // Target Devices:
11 // Tool Versions:
12 // Description:
13 //
14 // Dependencies:
15 //
16 // Revision:
17 // Revision 0.01 - File Created
18 // Additional Comments:
19 //
20 //////////////////////////////////////////////////////////////////////////////////
21
22
23 module Convolution_tb();
24
25     // ~~~~ imports ~~~~
26     import tb_utils_pkg::*;
27
28
29     // ~~~~ local param/allocs ~~~~
30     // ~~ ports ~~
31     logic                   clk;
32     logic [71:0]            mat;
33     logic                   mat_valid;
34     logic [7:0]             result1, result2;
35     logic                   result_valid1, result_valid2;
36     // ~~ consts ~~
37     localparam logic signed [7:0] K_BOX [0:8] =
38     '{
39         1, 1, 1,
40         1, 1, 1,
41         1, 1, 1
42     };
43
```

```
    localparam logic signed [7:0] K_SOBEL_X [0:8] =
    '{
        1, 0, -1,
        2, 0, -2,
        1, 0, -1
    };


    // ~~ testing ~~
    testcase tc;
    logic   [31:0]          tnum;


    // ~~~~ module instances ~~~~
    Convolution
    #(
        .FILTER(1'b0)
    )
    UUT_BOX
    (
        .CLK_i(clk),
        .MATRIX_i(mat),
        .MATRIX_VALID_i(mat_valid),
        .RESULT_o(result1),
        .RESULT_VALID_o(result_valid1)
    );

    Convolution
    #(
        .FILTER(1'b1)
    )
    UUT_SOBEL_X
    (
        .CLK_i(clk),
        .MATRIX_i(mat),
        .MATRIX_VALID_i(mat_valid),
        .RESULT_o(result2),
        .RESULT_VALID_o(result_valid2)
    );


    // ~~~~ heartbeat (10ns) ~~~~
    always begin
        #5;
        clk <= !clk;
    end


    // ~~~~ testing ~~~~
    initial begin

        // ~~ class instances ~~
        tc                  = new();

        // ~~ defaults ~~
        clk                 = 1'b1;
        mat                 = 72'h00_00_00___00_00_00___00_00_00;
        mat_valid           = 1'b1;
        #100;


        // ~~ TC1 ~~
        tc.new_test("1's Mat");
        tnum                    = tc.get_testnum();
            mat                 = 72'h01_01_01___01_01_01___01_01_01;
            #11;
            assert(result1 === {8'h01})                 else tc.err("BoxBlur result error");
            assert(result_valid1 === {1'b1})            else tc.err("BoxBlur result_valid error");
            assert(result2 === {8'h00})                 else tc.err("SobelX result error");
            assert(result_valid1 === {1'b1})            else tc.err("SobelX result_valid error");
            #9;
        tc.test_done();


        // ~~ TC2 ~~
        tc.new_test("Floor Mat");
        tnum                    = tc.get_testnum();
            mat                 = 72'h00_00_00___00_00_00___00_00_00;
            #11;
            assert(result1 === {8'h00})                 else tc.err("BoxBlur result error");
            assert(result_valid1 === {1'b1})            else tc.err("BoxBlur result_valid error");
            assert(result2 === {8'h00})                 else tc.err("SobelX result error");
            assert(result_valid2 === {1'b1})            else tc.err("SobelX result_valid error");
            #9;
        tc.test_done();

        // ~~ TC3 ~~
        tc.new_test("5's Mat");
        tnum                    = tc.get_testnum();
            mat                 = 72'h05_05_05___05_05_05___05_05_05;
            #11;
            assert(result1 === {8'h05})                 else tc.err("BoxBlur result error");
            assert(result_valid1 === {1'b1})            else tc.err("BoxBlur result_valid error");
            assert(result2 === {8'h00})                 else tc.err("SobelX result error");
            assert(result_valid2 === {1'b1})            else tc.err("SobelX result_valid error");
            #9;
        tc.test_done();

        // ~~ TC4 ~~
        tc.new_test("Ceiling Mat");
        tnum                    = tc.get_testnum();
            mat                 = 72'hFF_FF_FF___FF_FF_FF___FF_FF_FF;
            #11;
            assert(result1 === {8'hFF})                 else tc.err("BoxBlur result error");
            assert(result_valid1 === {1'b1})            else tc.err("BoxBlur result_valid error");
            assert(result2 === {8'h00})                 else tc.err("SobelX result error");
            assert(result_valid2 === {1'b1})            else tc.err("SobelX result_valid error");
            #9;
```

```
        tc.test_done();

        // ~~ TC5 ~~
        tc.new_test("Identity Mat");
        tnum                    = tc.get_testnum();
            mat                 = 72'h01_00_00___00_01_00___00_00_01;
            #11;
            assert(result1 === {8'h00})                 else tc.err("BoxBlur result error");
            assert(result_valid1 === {1'b1})            else tc.err("BoxBlur result_valid error");
            assert(result2 === {8'h00})                 else tc.err("SobelX result error");
            assert(result_valid2 === {1'b1})            else tc.err("SobelX result_valid error");
            #9;
        tc.test_done();

        // ~~ TC6 ~~
        tc.new_test("3x Identity Mat");
        tnum                    = tc.get_testnum();
            mat                 = 72'h03_00_00___00_03_00___00_00_03;
            #11;
            assert(result1 === {8'h01})                 else tc.err("BoxBlur result error");
            assert(result_valid1 === {1'b1})            else tc.err("BoxBlur result_valid error");
            assert(result2 === {8'h00})                 else tc.err("SobelX result error");
            assert(result_valid2 === {1'b1})            else tc.err("SobelX result_valid error");
            #9;
        tc.test_done();

        // ~~ TC7 ~~
        tc.new_test("Increasing Mat");
        tnum                    = tc.get_testnum();
            mat                 = 72'h01_02_03___04_05_06___07_08_09; // mat[0] = 09
            #11;
            assert(result1 === {8'h05})                 else tc.err("BoxBlur result error");
            assert(result_valid1 === {1'b1})            else tc.err("BoxBlur result_valid error");
            assert(result2 === {8'h08})                 else tc.err("SobelX result error");
            assert(result_valid2 === {1'b1})            else tc.err("SobelX result_valid error");
            #9;
        tc.test_done();

        // ~~ TC8 ~~
        tc.new_test("Increasing Mat Reversed");
        tnum                    = tc.get_testnum();
            mat                 = 72'h09_08_07___06_05_04___03_02_01;   // mat[0] = 01
            #11;
            assert(result1 === {8'h05})                 else tc.err("BoxBlur result error");
            assert(result_valid1 === {1'b1})            else tc.err("BoxBlur result_valid error");
            assert(result2 === {8'h00})                 else tc.err("SobelX result error");
                // should be -8 but floor clip
            assert(result_valid2 === {1'b1})            else tc.err("SobelX result_valid error");
            #9;
        tc.test_done();

        // ~~ TC9 ~~
        tc.new_test("Single-Centered 9");
        tnum                    = tc.get_testnum();
            mat                 = 72'h00_00_00___00_00_09___00_00_00;
            #11;
            assert(result1 === {8'h01})                 else tc.err("BoxBlur result error");
            assert(result_valid1 === {1'b1})            else tc.err("BoxBlur result_valid error");
            assert(result2 === {8'd18})                 else tc.err("SobelX result error");
            assert(result_valid2 === {1'b1})            else tc.err("SobelX result_valid error");
            #9;
        tc.test_done();

        // ~~ TC10 ~~
        tc.new_test("New data not ready");
        tnum                    = tc.get_testnum();
            mat                 = 72'h00_00_00___00_09_00___00_00_00;
            mat_valid   = 1'b0;
            #11;
            assert(result1 === {8'h01})                 else tc.err("BoxBlur result error");
            assert(result_valid1 === {1'b0})            else tc.err("BoxBlur result_valid error");
            assert(result2 === {8'd18})                 else tc.err("SobelX result error");
            assert(result_valid2 === {1'b0})            else tc.err("SobelX result_valid error");
            #9;
        tc.test_done();

        // ~~ TC11 ~~
        tc.new_test("Medium Range Numbers");
        tnum                    = tc.get_testnum();
            mat                 = 72'h81_81_81___81_81_81___81_81_81;
            mat_valid   = 1'b1;
            #11;
            assert(result1 === {8'h81})                 else tc.err("BoxBlur result error");
            assert(result_valid1 === {1'b1})            else tc.err("BoxBlur result_valid error");
            assert(result2 === {8'd00})                 else tc.err("SobelX result error");
            assert(result_valid2 === {1'b1})            else tc.err("SobelX result_valid error");
            #9;
        tc.test_done();

    end
endmodule
```

7.1.3 Image Control Unit

Listing 7.5: `ImageControlUnit.sv`

```
`timescale 1ns / 1ps
//////////////////////////////////////////////////////////////////////////////////
// Company:
// Engineer:
//
// Create Date: 09/01/2026 09:44:53 AM
// Design Name:
// Module Name: ImageControlUnit
// Project Name:
// Target Devices:
// Tool Versions:
```

```
// Description:
//
// Dependencies:
//
// Revision:
// Revision 0.01 - File Created
// Additional Comments:
//
//////////////////////////////////////////////////////////////////////////////////


module ImageControlUnit
#(parameter int unsigned PIXELWIDTH = 512,
            int unsigned PIXELHEIGHT = 512)
(
    input logic             CLK_i,
    input logic             RST_i,
    input logic             PIXEL_VALID_i,
    input logic     [7:0]   PIXEL_i,
    output logic            MATRIX_VALID_o,
    output logic    [71:0]  MATRIX_o,
    output logic            INTRR_o
);


    // ~~~~ local wires/param/alloc/const ~~~~
    typedef enum logic {IDLE, RD_BUFFER}    read_state;
    read_state rdState;

    localparam logic [11:0] pixelsInThreeRows = PIXELWIDTH * 3;

    // ~~ input pixel vars ~~
    logic           [8:0]   pixelCounter;       // index for pixel in row (designed to rollover)
    logic           [1:0]   currWrLineBuff;     // indicates how many rows(0-3) we've filled w/ pixels (designed to rollover)
    logic           [3:0]   validInput;         // dictates which LB gets to read


    // ~~ output pixel vars ~~
    logic           [8:0]   windowCounter;      // kernel position on the width-axis
    logic           [1:0]   currRdLineBuff;     // kernel row grouping selector
    logic           [3:0]   validOutput;        // pairs w/ above to enable specific LBs to output their 3 pixels
    logic           [11:0]  currBufferedPixels; // number of pixels currently in LBs


    logic           [23:0]  lb0_o;
    logic           [23:0]  lb1_o;
    logic           [23:0]  lb2_o;
    logic           [23:0]  lb3_o;


    // ~~~~ instances ~~~~
    LineBuffer #(PIXELWIDTH,PIXELHEIGHT) lb00
    (
        .CLK_i          (CLK_i),
        .RST_i          (RST_i),
        .DIN_i          (PIXEL_i),
        .DIN_AVAIL_i    (validInput[0]),
        .DOUT_AVAIL_i   (validOutput[0]),
        .DOUT_o         (lb0_o)
    );

    LineBuffer #(PIXELWIDTH,PIXELHEIGHT) lb01
    (
        .CLK_i          (CLK_i),
        .RST_i          (RST_i),
        .DIN_i          (PIXEL_i),
        .DIN_AVAIL_i    (validInput[1]),
        .DOUT_AVAIL_i   (validOutput[1]),
        .DOUT_o         (lb1_o)
    );

    LineBuffer #(PIXELWIDTH,PIXELHEIGHT) lb02
    (
        .CLK_i          (CLK_i),
        .RST_i          (RST_i),
        .DIN_i          (PIXEL_i),
        .DIN_AVAIL_i    (validInput[2]),
        .DOUT_AVAIL_i   (validOutput[2]),
        .DOUT_o         (lb2_o)
    );

    LineBuffer #(PIXELWIDTH,PIXELHEIGHT) lb03
    (
        .CLK_i          (CLK_i),
        .RST_i          (RST_i),
        .DIN_i          (PIXEL_i),
        .DIN_AVAIL_i    (validInput[3]),
        .DOUT_AVAIL_i   (validOutput[3]),
        .DOUT_o         (lb3_o)
    );


    // ~~~~ async logic ~~~~
    assign validInput       = {3'b000, PIXEL_VALID_i} << currWrLineBuff;

    always_comb begin
        case(currRdLineBuff)
            2'b00 : begin
                MATRIX_o    = {lb2_o, lb1_o, lb0_o};
                validOutput = {1'b0, {3{MATRIX_VALID_o}}};
            end

            2'b01 : begin
                MATRIX_o    = {lb3_o, lb2_o, lb1_o};
                validOutput = {{3{MATRIX_VALID_o}}, 1'b0};
            end
```

```
118
119             2'b10 : begin
120                 MATRIX_o     = {lb0_o, lb3_o, lb2_o};
121                 validOutput = {{2{MATRIX_VALID_o}}, 1'b0, MATRIX_VALID_o};
122             end
123
124             2'b11 : begin
125                 MATRIX_o     = {lb1_o, lb0_o, lb3_o};
126                 validOutput = {MATRIX_VALID_o, 1'b0, {2{MATRIX_VALID_o}}};
127             end
128
129             default : begin
130                 MATRIX_o     = 24'd0;
131                 validOutput = 4'd0;
132             end
133         endcase
134     end
135
136
137
138     // ~~~~ sync logic ~~~~
139     always_ff @(posedge CLK_i) begin
140
141         // ~~ priority 0 : reset clear ptrs and counters ~~
142         if(RST_i == 1'b1) begin
143             pixelCounter        <= 9'd0;
144             windowCounter       <= 9'd0;
145             currWrLineBuff      <= 2'd0;
146             currRdLineBuff      <= 2'd0;
147             currBufferedPixels  <= 12'd0;
148
149             rdState             <= IDLE;
150             MATRIX_VALID_o      <= 1'b0;
151             INTRR_o             <= 1'b0;
152         end
153
154         // ~~ priority 1 : read or write data ~~
155         else begin
156
157             // ~ process incoming pixel data ~
158             if(PIXEL_VALID_i == 1'b1) begin
159
160                 // move input pixel ptr to fill next spot
161                 pixelCounter            <= pixelCounter + 1;
162
163                 // max'd pixel counter rolls over and starts new row
164                 if(pixelCounter == (PIXELWIDTH-1)) begin
165                     currWrLineBuff      <= currWrLineBuff + 2'b01;
166                 end
167
168                 // buffer gains a pixel if only "deposit" occurs
169                 if(!MATRIX_VALID_o) begin currBufferedPixels    <= currBufferedPixels + 1; end
170             end
171
172
173             // ~ process outgoing window data ~
174             //if(writeOutEn) begin
175             if(MATRIX_VALID_o) begin
176
177                 // move output window ptr to next spot
178                 windowCounter           <= windowCounter + 1;
179
180                 // max'd window counter rolls over and starts new row
181                 if(windowCounter == (PIXELWIDTH-1)) begin
182                     currRdLineBuff      <= currRdLineBuff + 2'b01;
183                 end
184
185                 // buffer loses a pixel if only "withdraw" occurs
186                 if(PIXEL_VALID_i == 1'b0) begin currBufferedPixels  <= currBufferedPixels - 1; end
187             end
188
189         end
190
191         // ~~ FSM logic ~~
192         case(rdState)
193
194             // ~ IDLE : dont output til buffer enough rows for a full kernel pass ~
195             IDLE : begin
196                 if(currBufferedPixels >= pixelsInThreeRows) begin
197                     rdState         <= RD_BUFFER;
198                     MATRIX_VALID_o  <= 1'b1;
199                     INTRR_o         <= 1'b0;
200                 end
201             end
202
203             // ~ RD_BUFFER : output windows til the row end is reached ~
204             RD_BUFFER : begin
205                 if(windowCounter == PIXELWIDTH-1) begin
206                     rdState         <= IDLE;
207                     MATRIX_VALID_o  <= 1'b0;
208                     INTRR_o         <= 1'b1;
209                 end
210             end
211         endcase
212     end
213
214 endmodule
```

Listing 7.6: `ImageControlUnit_tb.sv`

```
1 `timescale 1ns / 1ps
2 //////////////////////////////////////////////////////////////////////////////////
3 // Company:
4 // Engineer:
5 //
6 // Create Date: 09/02/2026 01:38:35 PM
7 // Design Name:
```

```
// Module Name: ImageControlUnit_tb
// Project Name:
// Target Devices:
// Tool Versions:
// Description:
//
// Dependencies:
//
// Revision:
// Revision 0.01 - File Created
// Additional Comments:
//
//////////////////////////////////////////////////////////////////////////////////


module ImageControlUnit_tb();

    // ~~~~ imports ~~~~
    import tb_utils_pkg::*;


    // ~~~~ local param/allocs ~~~~
    // ~~ port ~~
    // ~ I ~
    logic                   clk;
    logic                   reset;
    logic                   pvalid;
    logic   [7:0]           pixel;
    // ~ O ~
    logic                   mvalid;
    logic   [71:0]          mat;

    // ~~ const ~~
    localparam int unsigned PIXELWIDTH = 512;

    // ~~ test ~~
    testcase tc;
    logic   [31:0]          tnum;
    logic   [71:0]          mat_ans;

    // ~~~~ instances ~~~~
    ImageControlUnit
    #(
        .PIXELWIDTH         (512),
        .PIXELHEIGHT        (512)
    )
    UUT
    (
        .CLK_i              (clk),
        .RST_i              (reset),
        .PIXEL_VALID_i      (pvalid),
        .PIXEL_i            (pixel),
        .MATRIX_VALID_o     (mvalid),
        .MATRIX_o           (mat)
    );

    // ~~~~ heartbeat (10ns) ~~~~
    always begin
        #5;
        clk <= !clk;
    end


    // ~~~~ testing ~~~~
    initial begin

        // ~~ class instances ~~
        tc                  = new();

        // ~~ defaults ~~
        clk                 = 1'b1;
        reset               = 1'b1;
        pvalid              = 1'b0;
        pixel               = 8'h00;
        #100;
        reset               = 1'b0;
        #10;

        // ~~ TC1 ~~
        tc.new_test("3 uniform rows only");
        tnum                = tc.get_testnum();
            uniform_fill_LineBuffer(255, 512, pixel, pvalid);   // row 0
            uniform_fill_LineBuffer(126, 512, pixel, pvalid);   // row 1
            uniform_fill_LineBuffer(7, 512, pixel, pvalid);     // row 2

            mat_ans = 72'h07_07_07___7E_7E_7E___FF_FF_FF;
            wait(mvalid);

            for(int i = 0; i < PIXELWIDTH; i++) begin
                // $display("|-%d", i);
                #5;
                assert(mvalid === 1'b1)                 else tc.err("mvalid HIGH error");
                assert(mat === mat_ans)                 else tc.err("mat error");
                #5;
            end

            // ~ run out of data ~
            #5;
            mat_ans = 72'hXX_XX_XX___07_07_07___7E_7E_7E;
            assert(mvalid === 1'b0)                     else tc.err("mvalid ending error");
            assert(mat === mat_ans)                     else tc.err("mat ending error");
            #5;
        tc.test_done();

        // ~ clear buffer ~
        uniform_fill_LineBuffer(0, PIXELWIDTH*1, pixel, pvalid);        // r3, r0, r1, r2, r3


```

```
116
117         // ~~ TC2 ~~
118         // Note : Kernel is on R(1,2,3)
119         // Note : wPtr is on R0
120         tc.new_test("blank");
121         tnum                = tc.get_testnum();
122
123         tc.test_done();
124
125
126     end
127
128
129
130
131     // ~~~~ utility tasks ~~~~
132     task automatic increment_fill_LineBuffer
133     (
134         input int unsigned  startNum,
135         input int unsigned  buffSize,
136         ref logic [7:0]     pxl,
137         ref logic           writeEn
138     );
139
140         writeEn = 1'b1;
141         for(int i = 0; i < buffSize; i++) begin
142             pxl = startNum + i;
143             #10;
144         end
145         writeEn = 1'b0;
146         pxl     = 8'h00;
147     endtask
148
149     task automatic uniform_fill_LineBuffer
150     (
151         input int unsigned  num,
152         input int unsigned  buffSize,
153         ref logic [7:0]     pxl,
154         ref logic           writeEn
155     );
156
157         writeEn = 1'b1;
158         for(int unsigned i = 0; i < buffSize; i++) begin
159             pxl = num;
160             #10;
161         end
162         writeEn = 1'b0;
163         pxl     = 8'h00;
164     endtask
165
166 endmodule
```

Chapter 8

References

Youtube Inspiration